SOMERSET

Analyser System for Water Quality Monitoring and Data Transmission

BLOCK DIAGRAM:

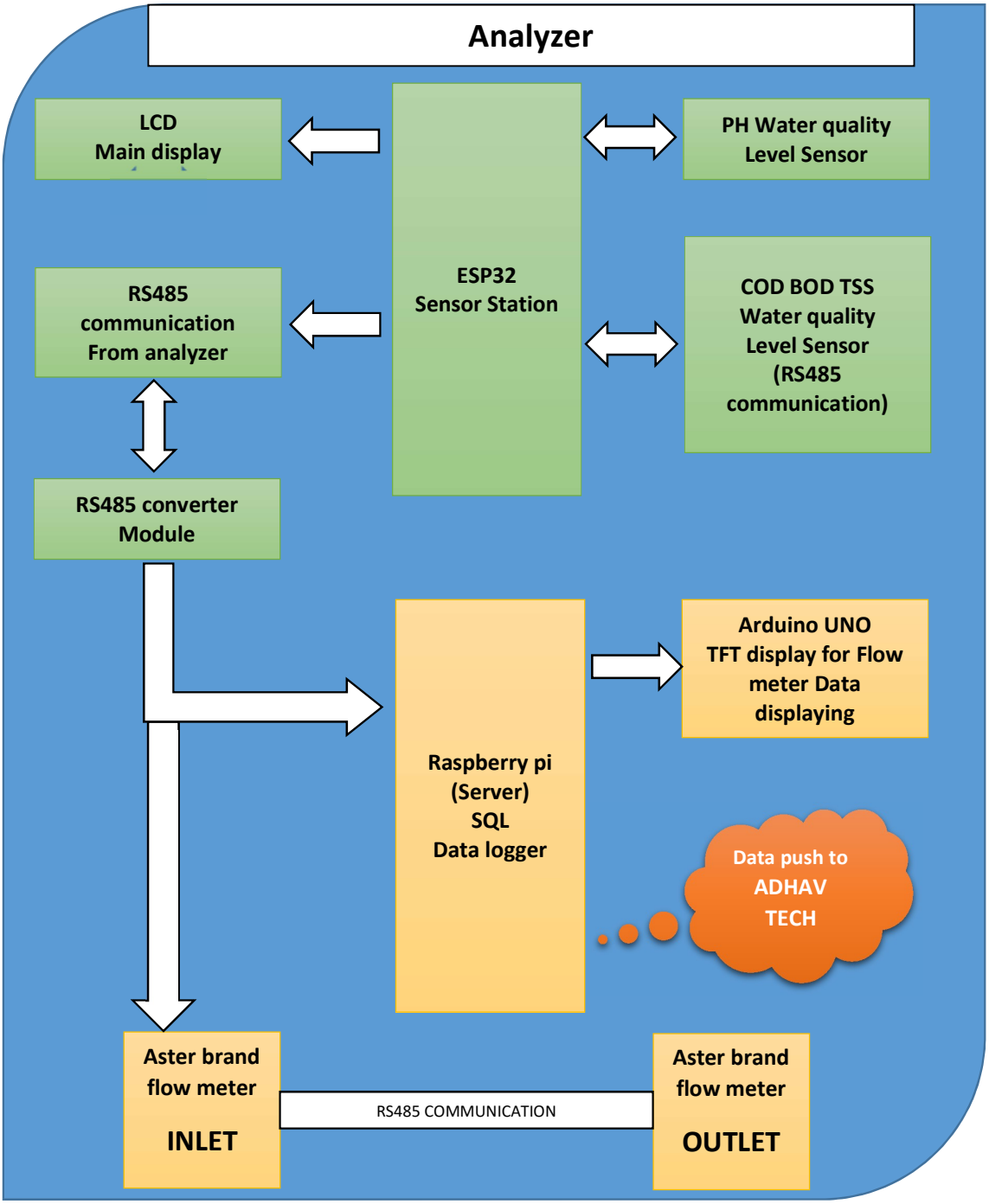


Figure 1: General Block diagram of Analyser

NOTES:

Scope of the projects

QUICK ENGINEERING GOWATER PVT LTD ADHAV TECH

SOMERSET

Analyser System for Water Quality Monitoring and Data Transmission

Hardware images:



Figure 2: Front view of installed analyser image (original)



Figure 3: inner circuit images

Software Implementation

1.1 Raspberry Pi program

```
#updated on 28.09.2023
import minimalmodbus
import serial
import time
import struct
import json
import requests
from serial.tools import list_ports

cod = 0.0
bod = 0.0
tss = 0.0
ph = 0.0
tot1 = 0.0
flow1 = 0.0
tot2 = 0.0
flow2 = 0.0

flow11 = 0
tot11 = 0



# Function to automatically find the Arduino port on both Windows and Linux
def find_arduino_port():
    arduino_ports = [p.device for p in list_ports.comports() if 'Arduino' in p.description]
    if not arduino_ports:
        arduino_ports = [p.device for p in list_ports.comports() if 'ttyACM' in p.device]
        if not arduino_ports:
```

```
        raise IOError("No Arduino found. Check if it's connected.")
    return arduino_ports[0]
try:
# Automatically find the Arduino port
    arduino_port = find_arduino_port()


# port name, slave address (in decimal)
    ser = serial.Serial(arduino_port, 9600, timeout=1)
except IOError as e:
    print(e)
    ser = None



an_instrument = minimalmodbus.Instrument('/dev/ttyUSB0', 1)
fm1_instrument = minimalmodbus.Instrument('/dev/ttyUSB0', 2)
fm2_instrument = minimalmodbus.Instrument('/dev/ttyUSB0', 3)
an_instrument.serial.baudrate = fm1_instrument.serial.baudrate = fm2_instrument.serial.baudrate
= 9600
an_instrument.serial.bytesize = fm1_instrument.serial.bytesize = fm2_instrument.serial.bytesize = 8
an_instrument.serial.stopbits = fm1_instrument.serial.stopbits = fm2_instrument.serial.stopbits = 1
an_instrument.serial.timeout = fm1_instrument.serial.timeout  = fm2_instrument.serial.timeout  = 1
an_instrument.mode = fm1_instrument.mode = fm2_instrument.mode =
minimalmodbus.MODE_RTU
an_instrument.clear_buffers_before_each_transaction =
fm1_instrument.clear_buffers_before_each_transaction =
fm2_instrument.clear_buffers_before_each_transaction = True
an_instrument.debug = fm1_instrument.debug = fm2_instrument.debug = True



def analyzer_cod():
    try:
        reg0 = an_instrument.read_register(0, 0, 3)
```

```
    reg1 = an_instrument.read_register(1, 0, 3)

    regset1 = hex((reg1<<16) | reg0)[2:]

    regset1_str = str(regset1)

    f_cod = struct.unpack('>f', bytes.fromhex(regset1_str))[0]

    cod = round(f_cod, 2)

    print("The COD HEX is : ", regset1_str)

    print("The COD Value is : ", cod)

    cod_data = "http://localhost/push/push.php?id=cod&cod={}".format(cod)

    update_cod_api = requests.get(cod_data)

    print(cod_data)

    print(update_cod_api.status_code)

    an_instrument.serial.close()


  except (minimalmodbus.NoResponseError, minimalmodbus.InvalidResponseError, ValueError,
UnboundLocalError) as error:

    print("Error in response of cod")

    ##############################################

def analyzer_bod():

  try:

    reg2 = an_instrument.read_register(2, 0, 3)

    reg3 = an_instrument.read_register(3, 0, 3)

    regset2 = hex((reg3<<16) | reg2)[2:]

    regset2_str = str(regset2)

    f_bod = struct.unpack('>f', bytes.fromhex(regset2_str))[0]

    bod = round(f_bod, 2)

    print("The BOD HEX is : ", regset2_str)

    print("The BOD Value is : ", bod)

    bod_data = "http://localhost/push/push.php?id=bod&bod={}".format(bod)

    update_bod_api = requests.get(bod_data)

    print(bod_data)

    print(update_bod_api.status_code)
```

```
        an_instrument.serial.close()


    except (minimalmodbus.NoResponseError, minimalmodbus.InvalidResponseError, ValueError) as
error:
        print("Error in response of bod")
        ##########################################
def analyzer_tss():
    global tss  # Use global variable
    try:
        reg4 = an_instrument.read_register(4, 0, 3)
        reg5 = an_instrument.read_register(5, 0, 3)
        regset3 = hex((reg5<<16) | reg4)[2:]
        regset3_str = str(regset3)
        try:
            f_tss = struct.unpack('>f', bytes.fromhex(regset3_str))[0]
        except (struct.error, ValueError):
            f_tss = 0.0

        tss = round(f_tss, 2)
        print("The TSS HEX is : ", regset3_str)
        print("The TSS Value is : ", tss)
        tss_data = "http://localhost/push/push.php?id=tss&tss={}".format(tss)
        update_tss_api = requests.get(tss_data)
        print(tss_data)
        print(update_tss_api.status_code)
        an_instrument.serial.close()


    except (minimalmodbus.NoResponseError, minimalmodbus.InvalidResponseError) as error:
        print("Error in response of tss")
        ##########################################
def analyzer_ph():
```

```
    try:
        reg6 = an_instrument.read_register(6, 0, 3)
        reg7 = an_instrument.read_register(7, 0, 3)
        regset4 = hex((reg7<<16) | reg6)[2:]
        regset4_str = str(regset4)
        f_ph = struct.unpack('>f', bytes.fromhex(regset4_str))[0]
        ph = round(f_ph, 2)
        print("The PH HEX is : ", regset4_str)
        print("The PH Value is : ", ph)
        ph_data = "http://localhost/push/push.php?id=ph&ph={}".format(ph)
        update_ph_api = requests.get(ph_data)
        print(ph_data)
        print(update_ph_api.status_code)


        an_instrument.serial.close()


    except (minimalmodbus.NoResponseError, minimalmodbus.InvalidResponseError, ValueError) as
error:
        print("Error in response of ph")


def flowmeter1():
    try:
        response = fm1_instrument.read_registers(0, 10, 3)
        tot1 = ((response[3] << 16) | response[2])
        #tot1 = tot1
        #tot1 = "{:.1f}".format(tot1)
        flow1 = response[1]
        flow11 = flow1
        tot11 = tot1
        print("The inlet fm totalizer is : ", tot1, "LPH")
        print("The inlet fm flowrate is : ", flow1, "LPH")
```

```
        fm1_data =
"http://localhost/push/push.php?id=fm1&fmid1=001&tot1={}&flow1={}".format(tot1,flow1)
        update_fm1_api = requests.get(fm1_data)
        print(fm1_data)
        print(update_fm1_api.status_code)
        #time.sleep(2)
        sensor_name = "Flow"
        sensor_value = flow1
        send_sensor_data(sensor_name , sensor_value)
        #time.sleep(2)
        sensor_name = "Tot"
        sensor_value = tot1
        send_sensor_data(sensor_name , sensor_value)
        #time.sleep(2)
        fm1_instrument.serial.close()

    except (minimalmodbus.NoResponseError, minimalmodbus.InvalidResponseError) as error:
        print("Error in response of FM1")

def flowmeter2():
    try:
        response2 = fm2_instrument.read_registers(0, 8, 3)
        tot2 = ((response2[3] << 16) | response2[2])
        #tot2 = tot2
        #tot2 = "{:.1f}".format(tot2)
        flow2 = response2[1]
        print("The outlet fm totalizer is : ", tot2, "LPH")
        print("The outlet fm flowrate is : ", flow2, "LPH")

        fm2_data =
"http://localhost/push/push.php?id=fm2&fmid2=002&tot2={}&flow2={}".format(tot2,flow2)
```

```
        update_fm2_api = requests.get(fm2_data)
        print(fm2_data)
        print(update_fm2_api.status_code)
        #time.sleep(2)
        sensor_name = "Flow2"
        sensor_value = flow2
        send_sensor_data(sensor_name , sensor_value)
        #time.sleep(2)
        sensor_name = "Tot2"
        sensor_value = tot2
        send_sensor_data(sensor_name , sensor_value)
        #time.sleep(2)
        fm2_instrument.serial.close()



    except (minimalmodbus.NoResponseError, minimalmodbus.InvalidResponseError) as error:
        print("Error in response of FM2")


def send_sensor_data(sensor_name, sensor_value):
    data = f"{sensor_name}:{sensor_value}\n"
    ser.write(data.encode())


while True:
    analyzer_cod()
    analyzer_bod()
    analyzer_tss()
    analyzer_ph()
    time.sleep(5)
    flowmeter1()
    time.sleep(5)
    flowmeter2()
```

```
#send_sensor_data(sensor_name, sensor_value)


time.sleep(1)

time.sleep(20)
```

2.1 Arduino UNO program:

```
#include <Adafruit_GFX.h>
#include <MCUFRIEND_kbv.h>
#include <stdio.h>        //input output library
#include <stdint.h>       //integer type library
#include <string.h>       //string library
#include <Wire.h>
MCUFRIEND_kbv tft;

#define LOWFLASH (defined(__AVR_ATmega328P__) && defined(MCUFRIEND_KBV_H_))
#include "bitmap_RGB.h"
//#include "image.h"
#define BLACK   0x0000
#define BLUE    0x001F
#define RED     0xF800
#define GREEN   0x07E0
#define CYAN    0x07FF
#define MAGENTA 0xF81F
#define YELLOW  0xFFE0
#define WHITE   0xFFFF
#define GREY    0x8410
#define ORANGE  0xE880
float InletFlow;
float InletTot;
float OutletFlow;
float OutletTot;
float i=0;
float j=0;
int width = 320;
int height = 480;


void logo(int x, int y, int w, int h) {
    tft.drawRGBBitmap(x, y, gowatr_64x64, w, h);
    //tft.drawRGBBitmap(x+200, y, somerset_64x64, w, h);
}

void setup() {
  Serial.begin(9600); // Set the baud rate to match the Raspberry Pi
  uint16_t ID = tft.readID();
    Serial.print("TFT ID = 0x");
    Serial.println(ID, HEX);
    if (ID == 0xD3D3) ID = 0x9486; // write-only shield
    tft.begin(ID);
    tft.setRotation(1);            //PORTRAIT
    tft.fillScreen(BLACK);
    tft.setTextColor(YELLOW, BLACK);
    tft.setTextSize(2);
```

```
    // testing();
    tft.setTextSize(2);
    tft.setCursor(135, 25);
    tft.setTextColor(WHITE,BLACK);
    tft.print("Somerset STP Flowmeters");
    tft.setTextSize(2);
    tft.setCursor(135, 47);
    tft.print("(in LPH)");
    printUI();
}

void loop() {
  if (Serial.available() > 0) {
    String sensorData = Serial.readStringUntil('\n');
    // Parse received data
    int separatorIndex = sensorData.indexOf(':');
    if (separatorIndex != -1) {
      String sensorName = sensorData.substring(0, separatorIndex);
      if(sensorName.equals("Flow"))
      {
      float InletFlow = sensorData.substring(separatorIndex + 1).toFloat();
      Serial.print("Received Sensor: ");
      Serial.print(sensorName);
      Serial.print(", Value: ");
      Serial.println(InletFlow);
      tft.fillRect(8,179,220,27,BLACK);
      tft.setCursor(10, 180);
      tft.setTextSize(3);
      tft.print(InletFlow);
      }
      if(sensorName.equals("Tot"))
      {
      float InletTot = sensorData.substring(separatorIndex + 1).toFloat();
      Serial.print("Received Sensor: ");
      Serial.print(sensorName);
      Serial.print(", Value: ");
      Serial.println(InletTot);
      tft.fillRect(8,258,220,27,BLACK);
      tft.setCursor(10, 260);
      tft.setTextSize(3);
      tft.print(InletTot);
      }
////////////////////////////////////////////////////////////////
      if(sensorName.equals("Flow2"))
      {
      float OutletFlow = sensorData.substring(separatorIndex + 1).toFloat();
      Serial.print("Received Sensor: ");
      Serial.print(sensorName);
```

```
      Serial.print(", Value: ");
      Serial.println(OutletFlow);
      tft.fillRect(248,179,220,27,BLACK);
      tft.setCursor(249, 180);
      tft.setTextSize(3);
      tft.print(OutletFlow);
      }
      if(sensorName.equals("Tot2"))
      {
      float OutletTot = sensorData.substring(separatorIndex + 1).toFloat();
      Serial.print("Received Sensor: ");
      Serial.print(sensorName);
      Serial.print(", Value: ");
      Serial.println(OutletTot);
      tft.fillRect(248,258,220,27,BLACK);
      tft.setCursor(249, 260);
      tft.setTextSize(3);
      tft.print(OutletTot);
      delay(1000);
      }
    }
  }
}

void printUI()
{
tft.drawRoundRect(5,5,470,71,5,TFT_WHITE);
tft.drawRoundRect(6,6,470,71,5,TFT_WHITE);

tft.drawRoundRect(5,90,235,225,5,TFT_WHITE);
tft.drawRoundRect(6,91,235,225,5,TFT_WHITE);

tft.drawRoundRect(240,90,235,225,5,TFT_WHITE);
tft.drawRoundRect(241,91,235,225,5,TFT_WHITE);

logo(40, 10, 64, 64);

tft.fillRect(26,90,180,40,TFT_GREEN);
tft.fillRect(270,90,180,40,TFT_CYAN);

tft.setCursor(62,100);
tft.setTextColor(TFT_BLACK);
tft.setTextSize(3);
tft.print("INLET");

tft.setCursor(10,140);
tft.setTextColor(TFT_GREEN);
tft.setTextSize(4);
```

```
tft.print("F~");

tft.setCursor(10,215);
tft.setTextColor(TFT_GREEN);
tft.setTextSize(4);
tft.print("T+");
////////////////////////////////////////////////////////////////////

tft.setCursor(312,100);
tft.setTextColor(TFT_BLACK);
tft.setTextSize(3);
tft.print("OUTLET");

tft.setCursor(250,140);
tft.setTextColor(TFT_CYAN);
tft.setTextSize(4);
tft.print("F~");

tft.setCursor(250,215);
tft.setTextColor(TFT_CYAN);
tft.setTextSize(4);
tft.print("T+");

/*tft.setCursor(210,245);
tft.setTextColor(TFT_GREEN);
tft.setTextSize(2);
tft.print("+");*/

/*tft.setCursor(455,245);
tft.setTextColor(TFT_CYAN);
tft.setTextSize(2);
tft.print("+");*/
}
```

SOMERSET

Analyser System for Water Quality Monitoring and Data Transmission

Embedded Program Files:

1. Python Script:



Somerset_Final_Pi_Script.py

2. Arduino code:



arduino_display_cdoe.rar